



Trabajo Practico

Visualización de algoritmos de ordenamiento

Martín Oscar Basualdo, Aaron Joaquín Orbegozo, Rubén Ruiz Díaz Centurión, Federico Denuchi

Comisión 07 – Grupo 3

El trabajo práctico consiste en la implementación de algoritmos de ordenamiento y la visualización de los mismos desde el navegador. Los algoritmos de ordenamiento son un conjunto de instrucciones que recibe como entrada una lista para organizar sus elementos en una secuencia específica, como orden ascendente o descendente, numérico o alfabético. En este trabajo se completó el código del visualizador con tres algoritmos: “Bubble sort”, “Insertion sort”, “Merge sort”, “Quick sort” y “Selection sort”, cumpliendo con el contrato que utiliza el visualizador.

Bajo un modelo de ejecución paso a paso. Cada llamada a la función `step()` debe realizar la unidad mínima de trabajo (una comparación o un intercambio). Esto requiere utilizar variables globales para continuar el estado del algoritmo entre llamadas, simulando los bucles for tradicionales.

Bubble Sort (sort_bubble.py)

Concepto: Compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Repite este proceso hasta que no se necesiten más intercambios.

Funcionamiento paso a paso:

Inicialización (init): Copia los valores, establece n (tamaño), e inicializa i y j en 0

Paso (step):

Compara items[j] con items[j+1]

Si items[j] > items[j+1], los intercambia (swap=True)

Avanza j (siguiente par adyacente)

Cuando completa una pasada (i+1 == n), reduce n (el último elemento ya está ordenado) y reinicia Termina cuando n == 1 (todos los elementos ordenados)

Características: Simple pero ineficiente. Cada pasada "burbujea" el elemento más grande hacia el final.

```
items = []
n = 0
i = 0
j = 0
valoresIniciales = []
def startValues():
    global items, n, i, j, valoresIniciales
    items = list(valoresIniciales)
    n = len(items)
    i = 0
    j = 0
def init(vals):
    global valoresIniciales
    valoresIniciales = vals
    startValues()
def step():
    global items, n, i, j # TODO:
    a=j
    b=j+1
    done=False
    if items[a]>items[b]:
        aux=items[a]
        items[a]=items[b]
        items[b]=aux
        swap=True
    else:
        swap=False
    i=i+1
    j=j+1
    if i+1==n:
        n=n-1
        i=0
        j=0
    if n==1:
        done=True
    return {"a": a,"b": b,"swap": swap, "done": done}
```

Insertion Sort (sort insertion.py)

Concepto: Construye el array ordenado elemento por elemento, insertando cada nuevo elemento en su posición correcta dentro de la parte ya ordenada.

Funcionamiento paso a paso:

Inicialización (init): Comienza desde $i=1$ (el segundo elemento), $j=None$

Paso (step):

Si $j == None$, inicia el proceso de inserción para $items[i]$ (establece $j = i$)

Mientras $j > 0$ y $items[j-1] > items[j]$:

Hace UN swap adyacente entre $j-1$ y j

Decrementa j (desplaza hacia la izquierda)

Retorna el swap realizado

Cuando no hay más desplazamientos, avanza i y resetea $j=None$

Termina cuando $i \geq n$

Características: Eficiente para listas pequeñas o casi ordenadas. Va "insertando" cada elemento en su lugar correcto.

```
items = []
n = 0
i = 0
j = None
valoresIniciales = []
def startValues():
    global items, n, i, j, valoresIniciales
    items = list(valoresIniciales)
    n = len(items)
    i = 1
    j = None
def init(vals):
    global valoresIniciales
    valoresIniciales = vals
    startValues()
def step():
    global items, n, i, j
    if i >= n:
        return {"done": True}
    if j == None:
        j = i
    while j > 0 and items[j-1] > items[j]:
        aux = items[j-1]
        items[j-1] = items[j]
        items[j] = aux
        j=j-1
    return {"a": j, "b": j+1, "swap": True, "done": False}
i=i+1
j=None
return {"done": False}
```

Merge Sort (sort_merge.py)

Concepto: Divide recursivamente el array en mitades, las ordena por separado y luego las mezcla.
Implementación in-place con rotaciones.

Funcionamiento paso a paso:

Inicialización (init): Crea una pila con (0, n-1, "dividir")

Paso (step):

Fase "dividir": Divide intervalos en mitades y los apila (divide y conquistar)

Fase "mezclar": Mezcla dos subarrays ordenados

Compara elementos de ambos subarrays

Si el del subarray derecho es menor, inicia rotación

Fase "rotación": Mueve un elemento del subarray derecho hacia su posición en el izquierdo mediante swaps adyacentes consecutivos

Termina cuando la pila está vacía y no hay mezclas pendientes

Características: Algoritmo eficiente ($O(n \log n)$) pero complejo de visualizar. La versión in-place usa rotaciones para evitar arrays auxiliares.

```
valores = []
cantidad = 0
valoresIniciales = []
pila = []
en_mezcla = False
izq = der = medio = 0
puntero_izq = puntero_der = 0
rotando = False
indice_rotacion = 0

def startValues():
    global valores, cantidad, pila, en_mezcla, valoresIniciales
    global izq, der, medio, puntero_izq, puntero_der, rotando, indice_rotacion
    valores = list(valoresIniciales)
    cantidad = len(valores)
    pila = []
    if cantidad > 1:
        pila.append((0, cantidad - 1, "dividir"))

    en_mezcla = False
    izq = der = medio = 0
    puntero_izq = puntero_der = 0
    rotando = False
    indice_rotacion = 0

def init(vals):
    global valoresIniciales
    valoresIniciales = vals
    startValues()

def step():
    global valores, cantidad, pila, en_mezcla
```

```

global izq, der, medio, puntero_izq, puntero_der, rotando, indice_rotacion
if not pila and not en_mezcla and not rotando:
    return {"a": 0, "b": 0, "swap": False, "done": True}
if rotando:
    a = indice_rotacion - 1
    b = indice_rotacion
    valores[a], valores[b] = valores[b], valores[a]
    indice_rotacion -= 1
    if indice_rotacion == puntero_izq:
        rotando = False
        puntero_izq += 1
        medio += 1
        puntero_der += 1
    return {"a": a, "b": b, "swap": True, "done": False}
if en_mezcla:
    if puntero_izq > medio or puntero_der > der:
        en_mezcla = False
        return {"a": izq, "b": der, "swap": False, "done": False}
    if valores[puntero_izq] <= valores[puntero_der]:
        a = puntero_izq
        b = puntero_der
        puntero_izq += 1
        return {"a": a, "b": b, "swap": False, "done": False}
    else:
        indice_a_mover = puntero_der
        indice_rotacion = indice_a_mover
        rotando = True
        return {"a": puntero_izq, "b": puntero_der, "swap": False, "done": False}
if pila:
    izq, der, fase = pila.pop()
    if izq >= der:
        return {"a": izq, "b": der, "swap": False, "done": False}
    if fase == "dividir":
        medio = (izq + der) // 2
        pila.append((izq, der, "mezclar"))
        pila.append((medio + 1, der, "dividir"))
        pila.append((izq, medio, "dividir"))
        return {"a": izq, "b": der, "swap": False, "done": False}
    if fase == "mezclar":
        medio = (izq + der) // 2
        puntero_izq = izq
        puntero_der = medio + 1
        en_mezcla = True
        return {"a": izq, "b": der, "swap": False, "done": False}
return {"a": 0, "b": 0, "swap": False, "done": True}

```

Quick Sort (sort_quick.py)

Concepto: Elige un pivote, particiona el array de modo que elementos menores queden a la izquierda y mayores a la derecha, luego ordena recursivamente cada partición.

Funcionamiento paso a paso:

Inicialización (init): Apila el intervalo completo (0, n-1)

Paso (step):

Toma un intervalo de la pila y elige el último elemento como pivote

Fase de partición (Lomuto):

pos_menores marca la frontera de elementos menores al pivote explorador recorre el array

Si valores[explorador] < pivote, hace swap con pos_menores y avanza la frontera

Si valores[explorador] >= pivote, solo avanza el explorador

Al terminar el recorrido, coloca el pivote en su posición final (swap con pos_menores)

Apila los subintervalos izquierdo y derecho del pivote

Termina cuando no quedan intervalos por procesar

Características: Muy eficiente en promedio ($O(n \log n)$). El esquema de Lomuto es simple, pero realiza más swaps que otras variantes.

```
valores = []
cantidad = 0
pila = []
en_particion = False           # ¿estamos particionando un intervalo?
izq = der = 0                  # límites del intervalo actual
indice_pivote = 0               # índice donde está el pivote
valor_pivote = None             # valor del pivote (para comparar)
pos_menores = 0                 # puntero 'i' del esquema de Lomuto
explorador = 0                  # puntero 'j' del esquema de Lomuto
valoresIniciales = []
```

```
def startValues():
    global valores, cantidad, pila, valoresIniciales
    global en_particion, izq, der, indice_pivote, valor_pivote
    global pos_menores, explorador
    valores = list(valoresIniciales)
    cantidad = len(valores)
    pila = []
    if cantidad > 1:
        pila.append((0, cantidad - 1))
    en_particion = False
    izq = der = indice_pivote = pos_menores = explorador = 0
    valor_pivote = None
```

```
def init(vals):
    global valoresIniciales
    valoresIniciales = vals
    startValues()
```

```

def step():
    global valores, pila, cantidad
    global en_particion, izq, der, indice_pivote, valor_pivote
    global pos_menores, explorador
    if not pila and not en_particion:
        return {"a": 0, "b": 0, "swap": False, "done": True}

    if en_particion:
        if explorador < der:
            a = explorador
            b = indice_pivote
            if valores[explorador] < valor_pivote:
                valores[pos_menores], valores[explorador] = (
                    valores[explorador],
                    valores[pos_menores],
                )
                intercambiado_a = pos_menores
                intercambiado_b = explorador
                pos_menores += 1
                explorador += 1
                return {"a": intercambiado_a, "b": intercambiado_b, "swap": True, "done": False}
            else:
                explorador += 1
                return {"a": a, "b": b, "swap": False, "done": False}
        if indice_pivote != pos_menores:
            valores[pos_menores], valores[indice_pivote] = (
                valores[indice_pivote],
                valores[pos_menores],
            )
            intercambiado_a = pos_menores
            intercambiado_b = indice_pivote
            posicion_pivote_final = pos_menores
            en_particion = False
            if posicion_pivote_final + 1 < der:
                pila.append((posicion_pivote_final + 1, der))

            if izq < posicion_pivote_final - 1:
                pila.append((izq, posicion_pivote_final - 1))

            return {"a": intercambiado_a, "b": intercambiado_b, "swap": True, "done": False}
        else:
            posicion_pivote_final = pos_menores
            en_particion = False
            if posicion_pivote_final + 1 < der:
                pila.append((posicion_pivote_final + 1, der))

            if izq < posicion_pivote_final - 1:
                pila.append((izq, posicion_pivote_final - 1))
            return {"a": izq, "b": der, "swap": False, "done": False}

    if pila:
        izq, der = pila.pop()

    if izq >= der:
        return {"a": izq, "b": der, "swap": False, "done": False}

```

```

    indice_pivote = der
    valor_pivote = valores[indice_pivote]
    pos_menores = izq
    explorador = izq
    en_particion = True
    return {"a": izq, "b": der, "swap": False, "done": False}
return {"a": 0, "b": 0, "swap": False, "done": True}

```

Selection Sort (sort_selection.py)

Concepto: Divide el array en parte ordenada y no ordenada. Repetidamente busca el mínimo en la parte no ordenada y lo coloca al final de la parte ordenada.

Funcionamiento paso a paso:

Inicialización (init): $i=0$ (inicio de parte no ordenada), $j=i+1$, $\text{min_idx}=i$, $\text{fase}=\text{"buscar"}$

Paso (step):

Fase "buscar":

Recorre desde j hasta el final buscando el elemento mínimo

Actualiza min_idx cuando encuentra uno menor

Al llegar al final, cambia a fase "swap"

Fase "swap":

Intercambia $\text{items}[i]$ con $\text{items}[\text{min_idx}]$ (si son diferentes)

Avanza i , reinicia $j=i+1$ y $\text{min_idx}=i$

Vuelve a fase "buscar"

Termina cuando $i+1 == n$

```

items = []
n = 0
i = 0          # cabeza de la parte no ordenada
j = 0          # cursor que recorre y busca el mínimo
min_idx = 0    # índice del mínimo de la pasada actual
fase = "buscar" # "buscar" | "swap"
valoresIniciales = []

def startValues():
    global items, n, i, j, min_idx, fase, valoresIniciales
    items = list(valoresIniciales)
    n = len(items)
    i = 0
    j = i + 1
    min_idx = i
    fase = "buscar"
def init(vals):
    global valoresIniciales
    valoresIniciales = vals
    startValues()
def step():

```

```

global items, n, i, j, min_idx, fase
if(i+1==n):
    return {"done":True}
if(fase == "buscar"):
    if(items[j]<items[min_idx]):
        min_idx = j
        return {"a": min_idx, "b": j, "swap": False, "done": False}
    j=j+1
    if(j>=n):
        fase="swap"
        return {"a": min_idx, "b": j, "swap": False, "done":False}
if(fase=="swap"):

    minAux = min_idx # guardamos el índice del mínimo encontrado
    iAux = i # guardamos la posición donde debe colocarse
    if(items[minAux]!=items[iAux]):
        itemsAux = items[i]          # guardamos temporalmente items[i]
        items[min_idx] = items[i]    # movemos el valor actual a la posición del mínimo
        items[i] = itemsAux          # colocamos el mínimo en su lugar
        min_idx = i                  # el mínimo ahora quedó en la posición i
        return {"a": iAux, "b": minAux, "swap": True, "done": False}
    i = i+1                          # avanzamos a la siguiente posición a ordenar
    j = i+1                          # el nuevo recorrido empieza desde i+1
    min_idx = i                      # asumimos que el nuevo mínimo arranca en i
    fase = "buscar" # volvemos a buscar
    return {"done":False}

```

Dificultades generales:

Buscamos información y lo que encontrábamos estaban más enfocados en bucles y no en pasos de ordenamiento tendiendo a replicar el pseudo código de referencia que se encontraba en la función de step de ejemplo.

Al hacer uso de los algoritmos nos encontramos que al intentar resetear en mitad de ejecución y volver a iniciarlo, lo expresado posteriormente en pantalla no coincidía con el proceso actual desde un inicio, si no, que continuaba donde el anterior proceso había quedado, dando como resultado una disparidad entre lo visual y lo lógico. Se implementa un método en el que se tiene en cuenta los valores iniciales y se declara como función en la lógica visual, en donde los valores se resetean tanto en la lógica generada en Python como en lo mostrado en pantalla.